```
from google.colab import files
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

```
# Upload image
uploaded = files.upload()

# Get uploaded image name
image_path = list(uploaded.keys())[0]

# Load image
img = cv2.imread(image_path)
```

Choose Files imagesofcat.jpg

imagesofcat.jpg(image/jpeg) - 23667 bytes, last modified: 9/18/2026 - 100% done

```
Saving imagesofcat.jpg to imagesofcat.jpg
```

```
# Convert BGR to RGB for displaying
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

```
# Convert image to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

```
# Display original image
plt.figure(figsize=(6, 5))
plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis("off")
plt.show()
```




```
#Step 2 – Display Grayscale Image

plt.figure(figsize=(6, 5))

plt.imshow(gray, cmap="gray")
plt.title("Original Grayscale Image")
plt.axis("off")

plt.show()
```



```
# Apply Gaussian Blur
gaussian = cv2.GaussianBlur(gray, (5, 5), 0)

# Display result
plt.figure(figsize=(6, 5))

plt.imshow(gaussian, cmap="gray")
plt.title("Gaussian Blur")
plt.axis("off")

plt.show()
```




```
# Apply Median Filter
median = cv2.medianBlur(gray, 5)

# Display result
plt.figure(figsize=(6, 5))

plt.imshow(median, cmap="gray")
plt.title("Median Filter")
plt.axis("off")

plt.show()
```




```
# Apply Average / Mean Filter
average = cv2.blur(gray, (5, 5))

# Display result
plt.figure(figsize=(6, 5))

plt.imshow(average, cmap="gray")
plt.title("Average / Mean Filter")
plt.axis("off")

plt.show()
```




```
# Apply Laplacian Filter
laplacian = cv2.Laplacian(gray, cv2.CV_64F)

# Convert to absolute values
laplacian = cv2.convertScaleAbs(laplacian)

# Display result
plt.figure(figsize=(6, 5))

plt.imshow(laplacian, cmap="gray")
plt.title("Laplacian Filter")
plt.axis("off")

plt.show()
```




```
# Sobel X - Horizontal direction
sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)

# Convert to uint8
sobel_x = cv2.convertScaleAbs(sobel_x)

# Display
plt.figure(figsize=(6, 5))

plt.imshow(sobel_x, cmap="gray")
plt.title("Sobel X - Horizontal Edges")
plt.axis("off")

plt.show()
```




```
# Sobel Y - Vertical direction
sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)

# Convert to uint8
sobel_y = cv2.convertScaleAbs(sobel_y)

# Display
plt.figure(figsize=(6, 5))

plt.imshow(sobel_y, cmap="gray")
plt.title("Sobel Y - Vertical Edges")
plt.axis("off")

plt.show()
```




```
# Combine Sobel X and Sobel Y
sobel_combined = cv2.addWeighted(
    sobel_x, 0.5,
    sobel_y, 0.5,
    0
)

# Display
plt.figure(figsize=(6, 5))

plt.imshow(sobel_combined, cmap="gray")
plt.title("Combined Sobel Edge Detection")
plt.axis("off")

plt.show()
```

Combined Sobel Edge Detection



```
plt.figure(figsize=(15, 10))

# Original
plt.subplot(2, 4, 1)
plt.imshow(gray, cmap="gray")
plt.title("Original")
plt.axis("off")

# Gaussian
plt.subplot(2, 4, 2)
plt.imshow(gaussian, cmap="gray")
plt.title("Gaussian Blur")
plt.axis("off")

# Median
plt.subplot(2, 4, 3)
plt.imshow(median, cmap="gray")
plt.title("Median Filter")
plt.axis("off")

# Average
plt.subplot(2, 4, 4)
plt.imshow(average, cmap="gray")
plt.title("Average Filter")
plt.axis("off")
```